



TECNOLÓGICO
NACIONAL DE MÉXICO



**TECNOLÓGICO NACIONAL DE
MÉXICO**

**INSTITUTO TECNOLÓGICO
SUPERIOR DE HUICHAPAN**

MANUAL TÉCNICO

Alumno: Esmeralda Trejo Martínez

Profesor: Carlos Arturo Espinoza Galicia

Tecnologías Emergentes

Semestre: 8

Grupo: 1

Contenido

Introducción	2
Objetivo del proyecto	2
Objetivo general	2
Objetivos específicos	2
Materiales utilizados	4
Diagrama de Conexión	5
Instalación del Entorno	13
Instalación del Arduino IDE	13
Instalación de Ubuntu Server en Maquina Virtual	17
Instalación de Docker en Ubuntu Server	26
Implementacion de contenedores y infraestructura	31
1. La Base: Docker y Docker Compose	33
2. Mosquitto (El Broker MQTT) - <i>El que recibe</i>	34
3. InfluxDB (La Base de Datos de Series de Tiempo) - <i>El que guarda</i>	34
4. Grafana (La Interfaz Gráfica) - <i>El que muestra</i>	34
Herramienta utilizada	36
Usuario telegraf (lector/recolector de datos):	36
Usuarios de los dispositivos ESP32:	37
Verificacion de usuarios creados	37
Ajuste de permisos del archivo de contraseñas	38
Lista de Control de Acceso (ACL)	39
Montaje de volúmenes en Docker	41
Aplicacion de cambios	41
Reglas de Firewall con UFW	42
Puertos habilitados	43
Comandos utilizados para configurar el firewall	43

Introducción

La evolución de las Tecnologías de la Información y la Comunicación (TIC) ha impulsado el despliegue de soluciones basadas en el Internet de las Cosas (IoT), optimizando la supervisión y gestión de variables físicas en tiempo real. En los sectores agrícola y ambiental, esta capacidad es crítica; el monitoreo preciso de la humedad (suelo y ambiente), la temperatura y la iluminación determina el éxito en el desarrollo biológico de los cultivos.

Bajo esta premisa, el presente manual documenta el diseño e implementación de un sistema IoT basado en el microcontrolador ESP32. El dispositivo integra sensores para la adquisición de datos, visualización local y la posterior transmisión a un servidor para su análisis. La estructura de este documento sigue una metodología modular, permitiendo un recorrido progresivo por las decisiones técnicas y fases de desarrollo del proyecto.

Objetivo del proyecto

Objetivo general

Diseñar e implementar un sistema IoT basado en el microcontrolador ESP32 que permita el monitoreo de variables ambientales en una planta, proporcionando alertas visuales y sonoras, así como el envío de información a un servidor para su almacenamiento en una base de datos de series temporales.

Objetivos específicos

- Medir la humedad del suelo mediante un sensor dedicado.
- Medir la temperatura y la humedad ambiental utilizando sensores digitales.
- Medir la intensidad lumínica presente en el entorno de la planta.
- Visualizar las lecturas de los sensores de forma local mediante indicadores.
- Implementar alertas visuales y sonoras cuando las condiciones de humedad del suelo no sean adecuadas.
- Enviar los datos recolectados a un servidor mediante el protocolo MQTT.
- Almacenar la información en una base de datos de series temporales para su análisis posterior.

Capítulo 1

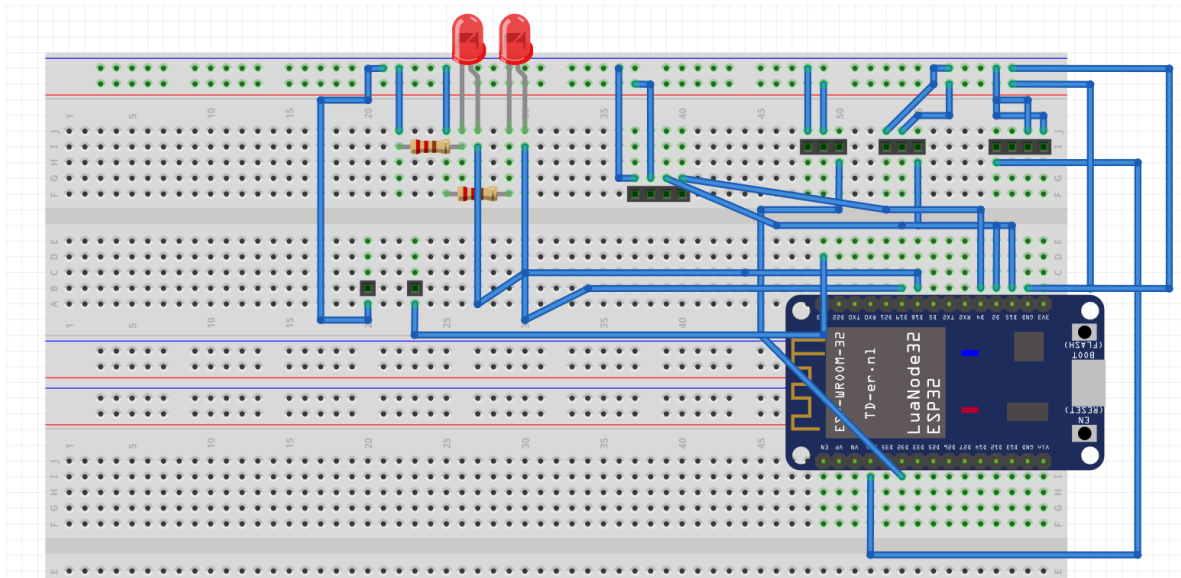
Materiales utilizados

Para el desarrollo del prototipo correspondiente al Capítulo 1, se emplearon los siguientes componentes electrónicos y materiales de apoyo. Los precios indicados corresponden a valores aproximados en el mercado mexicano y pueden variar según proveedor y disponibilidad.

No.	Componente	Especificación / Modelo	Cantidad	Precio Unitario Aproximado (MXN)
1	Microcontrolador	ESP32 DevKit V1	1	\$180 – \$250
2	Pantalla OLED	0.96" 128x64 I2C SSD1306	1	\$90 – \$130
3	Sensor temperatura y humedad	DHT11	1	\$40 – \$70
4	Sensor humedad de suelo	Sensor resistivo analógico	1	\$50 – \$80
5	Sensor de luz	LDR 5mm	1	\$5 – \$15
6	Botón pulsador	Push button 4 pines	1	\$5 – \$10
7	LED verde	5mm	1	\$3 – \$5
8	LED rojo	5mm	1	\$3 – \$5
9	Buzzer	Buzzer activo 5V	1	\$15 – \$30
10	Resistencias	220Ω, 10kΩ (aprox.)	Varias	\$1 – \$3 c/u
11	Protoboard	830 puntos	1	\$80 – \$120
12	Cables jumper	Macho–macho	1 set	\$40 – \$80

Diagrama de Conexión

La representación del hardware se realizó mediante **Fritzing**, emulando el montaje físico sobre protoboard. Ante la ausencia de ciertos componentes en la librería estándar del software, se emplearon elementos equivalentes —como *headers* macho-hembra— para garantizar una correspondencia exacta con las conexiones eléctricas del prototipo real.



Consideraciones generales del circuito

El sistema está basado en una placa ESP32, la cual actúa como unidad central de procesamiento. Desde ella se alimentan y controlan todos los sensores y actuadores del sistema.

Se establecieron las siguientes líneas principales en la protoboard:

- Línea de 3.3V proveniente del ESP32 para alimentar sensores y pantalla OLED.
- Línea común de GND (tierra), compartida por todos los componentes.

Es fundamental que todas las tierras (GND) estén conectadas en común para garantizar referencias de voltaje correctas y evitar lecturas erróneas.

Analisis deCodigo

1. Sección: Librerías y Dependencias

```
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#include "DHT.h"
#include <WiFi.h>
```

El sistema utiliza cinco librerías principales que permiten la comunicación con los componentes hardware:

- **Wire.h:** Protocolo I2C para comunicación con la pantalla OLED
- **Adafruit_GFX y Adafruit_SSD1306:** Control de la pantalla OLED de 128x64 píxeles
- **DHT.h:** Lectura del sensor de temperatura y humedad DHT11
- **WiFi.h:** Gestión de conectividad WiFi en modo estación (STA) y punto de acceso (AP)

2. Sección: Configuración de Hardware

2.1 Pantalla OLED Define los parámetros de la pantalla SSD1306 de 128x64 píxeles con dirección I2C 0x3C.

```
#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
#define OLED_RESET -1
#define SCREEN_ADDRESS 0x3C
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, OLED_RESET);
```

2.2 Sensor DHT11 Configurado en el pin GPIO 15 para lectura de temperatura y humedad ambiental.

```
#define DHTPIN 15
#define DHTTYPE DHT11
DHT dht(DHTPIN, DHTTYPE);
```

2.3 Pines de Entrada Analógica

- **Pin 32:** Sensor de humedad del suelo (lectura de 0-4095)
- **Pin 34:** Sensor LDR de luz ambiental (lectura de 0-4095)
- **Pin 16:** Botón físico con resistencia pull-up para activar modo AP

```
#define LED_VERDE 18  
#define LED_ROJO 19  
#define BUZZER 23
```

2.4 Pines de Salida

- **Pin 18:** LED verde (condiciones normales)
- **Pin 19:** LED rojo (alerta de humedad baja)
- **Pin 23:** Buzzer piezoactivo (alerta sonora)

```
#define LED_VERDE 18  
#define LED_ROJO 19  
#define BUZZER 23
```

3. Sección: Sistema de Calibración del Sensor de Suelo

```
#define HUMEDAD_SECO 3200  
#define HUMEDAD_HUMEDO 1400  
#define HUMEDAD_MINIMA 40
```

El sensor capacitivo de humedad del suelo requiere calibración previa:

- **Valor en seco:** 3200 (sensor al aire libre)
- **Valor en húmedo:** 1400 (sensor sumergido en agua)
- **Umbral de alerta:** 40% (por debajo activa alarmas)

Estos valores se mapean linealmente para obtener un porcentaje de 0-100%.

4. Sección: Sistema de Alerta Sonora Inteligente

```
unsigned long tiempoAnteriorBuzzer = 0;
bool buzzerActivo = false;
const unsigned long TIEMPO_SONIDO = 300;
const unsigned long TIEMPO_SILENCIO = 3000;
const int FRECUENCIA_BUZZER = 4000;
```

El buzzer opera con un patrón intermitente no bloqueante:

- Emite un tono de 4 kHz durante 300 ms
- Permanece en silencio durante 3 segundos
- Utiliza la función millis() para evitar bloquear el loop() principal
- Solo se activa cuando la humedad del suelo está por debajo del 40%

5. Sección: Configuración Dual WiFi (AP + STA)

Modo Punto de Acceso (AP)

- SSID: "ESP32_AP_Monitor"
- Contraseña: "12345678"
- Se activa manualmente mediante botón físico
- Permite configuración o acceso directo al dispositivo

```
const char* ssidAP = "ESP32_AP_Monitor";
const char* passwordAP = "12345678";
bool apActivo = false;
```

Modo Estación (STA)

- SSID predefinido: "Prueba"
- Contraseña: "123456789"
- Conexión automática al iniciar
- Timeout de 10 segundos, luego pasa a modo offline

```
const char* ssidCasa = "Prueba";  
const char* passCasa = "123456789";  
  
String deviceID = "nodo_kevin_monitor";
```

Identificador del Dispositivo

- deviceID = "nodo_kevin_monitor" para identificación en redes con múltiples nodos

6. Sección: Funciones del Sistema WiFi

iniciarAP() Activa el modo dual WiFi (AP_STA) permitiendo que el ESP32 funcione como punto de acceso mientras mantiene conexión a la red principal.

```
void iniciarAP() {  
    WiFi.mode(WIFI_AP_STA);  
    WiFi.softAP(ssidAP, passwordAP);  
    Serial.println("Access Point INICIADO");  
    Serial.print("IP AP: ");  
    Serial.println(WiFi.softAPIP());  
    apActivo = true;  
}
```

detenerAP() Desactiva el punto de acceso cuando el botón es liberado, optimizando el consumo energético.

```
void detenerAP() {  
    WiFi.softAPdisconnect(true);  
    Serial.println("Access Point DETENIDO");  
    apActivo = false;  
}
```

conectarWiFiCasa() Intenta conexión automática a la red principal durante 10 segundos. Si falla, continúa en modo offline sin interrumpir las funciones principales del monitor.

```
void conectarWiFiCasa() {
    Serial.print("Conectando a WiFi: ");
    Serial.println(ssidCasa);

    WiFi.mode(WIFI_STA);
    WiFi.begin(ssidCasa, passCasa);

    unsigned long inicio = millis();

    while (WiFi.status() != WL_CONNECTED && millis() - inicio < 10000) {
        delay(500);
        Serial.print(".");
    }

    if (WiFi.status() == WL_CONNECTED) {
        Serial.println("\nWiFi CONECTADO!");
        Serial.print("IP: ");
        Serial.println(WiFi.localIP());
    } else {
        Serial.println("\nModo OFFLINE");
    }
}
```

7. Sección: Sistema de Alertas Visuales y Sonoras

manejarAlertas(int humedadSuelo) Evalúa el nivel de humedad del suelo y gestiona tres actuadores:

- **Humedad $\geq$ 40%:** LED verde encendido, LED rojo y buzzer apagados
- **Humedad $<$ 40%:** LED rojo encendido, LED verde apagado, buzzer activado

```
void manejarAlertas(int humedadSuelo) {
    alertaSuelo = (humedadSuelo < HUMEDAD_MINIMA);

    if (!alertaSuelo) {
        digitalWrite(LED_VERDE, HIGH);
        digitalWrite(LED_ROJO, LOW);
        noTone(BUZZER);
        buzzerActivo = false;
    } else {
        digitalWrite(LED_VERDE, LOW);
        digitalWrite(LED_ROJO, HIGH);
    }
}
```

actualizarBuzzer() Implementa el patrón intermitente del buzzer utilizando máquina de estados:

- Verifica si hay alerta activa
- Controla los tiempos de sonido y silencio sin usar delay()
- Permite que el resto del sistema funcione normalmente

```
void actualizarBuzzer() {
    if (!alertaSuelo) return;

    unsigned long tiempoActual = millis();

    if (buzzerActivo && (tiempoActual - tiempoAnteriorBuzzer >= TIEMPO_SONIDO)) {
        noTone(BUZZER);
        buzzerActivo = false;
        tiempoAnteriorBuzzer = tiempoActual;
    }
    else if (!buzzerActivo && (tiempoActual - tiempoAnteriorBuzzer >= TIEMPO_SILENCIO)) {
        tone(BUZZER, FRECUENCIA_BUZZER);
        buzzerActivo = true;
        tiempoAnteriorBuzzer = tiempoActual;
    }
}
```

8. Sección: Configuración Inicial (setup)

Secuencia de inicialización:

1. Comunicación serial a 115200 baudios
2. Inicialización I2C en pines GPIO 4 (SDA) y GPIO 2 (SCL)
3. Activación del sensor DHT11
4. Configuración de pines digitales y analógicos
5. Intento de conexión WiFi (no bloqueante)
6. Inicialización y prueba de pantalla OLED
7. Pantalla de bienvenida durante 2 segundos

```

void setup() {

  Serial.begin(115200);
  Wire.begin(4, 2);

  dht.begin();

  pinMode(BUTTON_PIN, INPUT_PULLUP);
  pinMode(LDR_PIN, INPUT);
  pinMode(SOIL_PIN, INPUT);

  pinMode(LED_VERDE, OUTPUT);
  pinMode(LED_ROJO, OUTPUT);
  pinMode(BUZZER, OUTPUT);

  digitalWrite(LED_VERDE, LOW);
  digitalWrite(LED_ROJO, LOW);

  conectarWiFiCasa();

  if (!display.begin(SSD1306_SWITCHCAPVCC, SCREEN_ADDRESS)) {
    while (true);
  }

  display.setTextColor(SSD1306_WHITE);
  display.setTextSize(1);

```

Sección 9: Bucle Principal (loop)

El bucle principal ejecuta un ciclo continuo de monitoreo cada 2 segundos, realizando las siguientes operaciones de forma secuencial y no bloqueante:

Adquisición de Datos de Sensores El sistema realiza la lectura de todos los sensores conectados:

- Sensor DHT11 para temperatura y humedad ambiental, con validación de errores
- Sensor capacitivo de humedad del suelo, aplicando calibración y mapeo a porcentaje
- Sensor LDR para nivel de luminosidad ambiental
- Estado del botón físico para control de funciones WiFi

```

void loop() {

  float humedadAire = dht.readHumidity();
  float temperatura = dht.readTemperature();

  if (isnan(humedadAire) || isnan(temperatura)) {
    humedadAire = 0;
    temperatura = 0;
  }

  int soilValue = analogRead(SOIL_PIN);
  int soilPercent = map(soilValue, HUMEDAD_SECO, HUMEDAD_HUMEDO, 0, 100);
  soilPercent = constrain(soilPercent, 0, 100);

  int ldrValue = map(analogRead(LDR_PIN), 0, 4095, 100, 0);
  ldrValue = constrain(ldrValue, 0, 100);

  bool botonPresionado = digitalRead(BUTTON_PIN) == LOW;

  if (botonPresionado && !apActivo) iniciarAP();
  else if (!botonPresionado && apActivo) detenerAP();

  manejarAlertas(soilPercent);
  actualizarBuzzer();

```

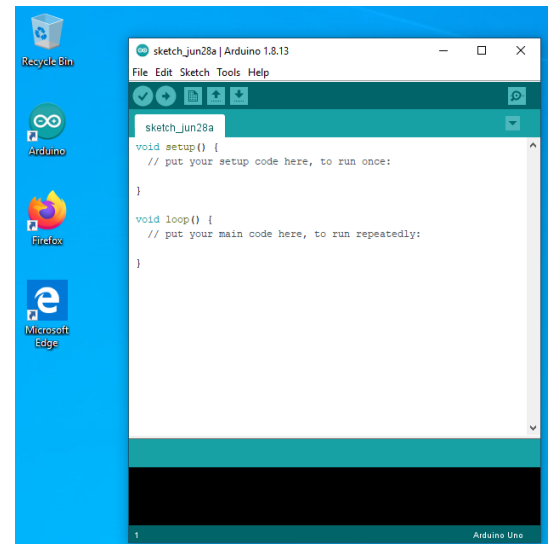
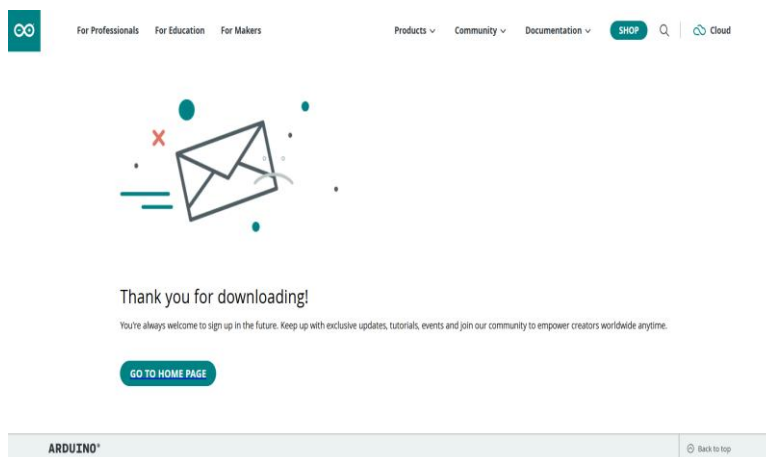
Instalación del Entorno

Instalación del Arduino IDE

Para la programación del microcontrolador ESP32 se utilizó el Arduino IDE, un entorno de desarrollo ampliamente utilizado en proyectos de electrónica e IoT. El Arduino IDE fue instalado en el sistema operativo Windows, permitiendo la escritura, compilación y carga de programas directamente al microcontrolador.

Este entorno facilita la integración de librerías externas y ofrece herramientas como el monitor serial, el cual resulta fundamental para la depuración y validación de lecturas durante las pruebas iniciales del proyecto.

Vamos a la página arduino.cc y bajamos la última versión para Windows (o el sistema operativo de su preferencia) y la instalamos.



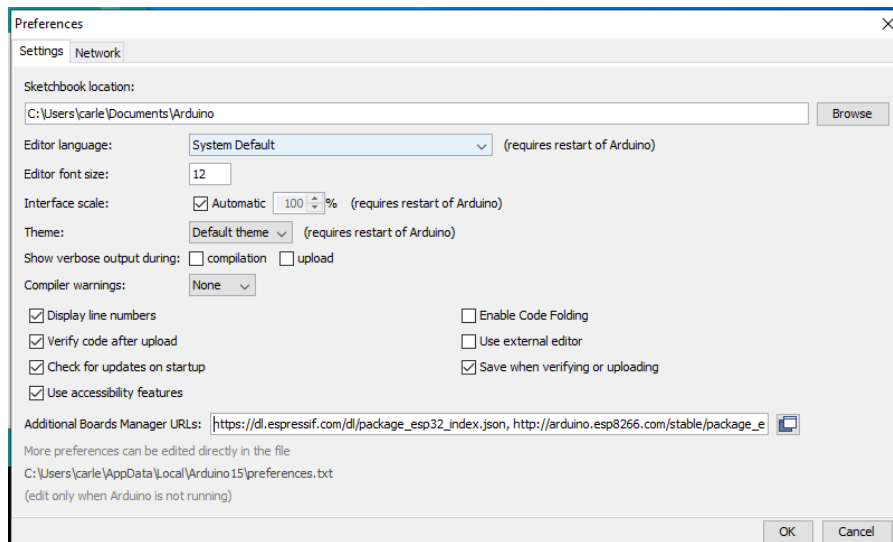
Instalación de los controladores ESP32 en el Arduino IDE

Abra el Arduino IDE, vaya a Archivo, Preferencias.

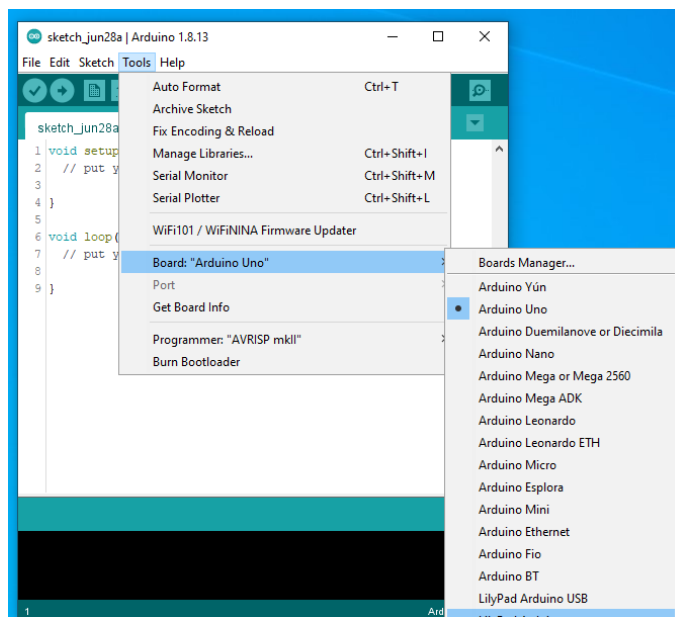
Ponga en la pestaña URL adicionales de gestor de placas las direcciones del fabricante ESPRESSIF:

- https://dl.espressif.com/dl/package_esp32_index.json,
- http://arduino.esp8266.com/stable/package_esp8266com_index.json

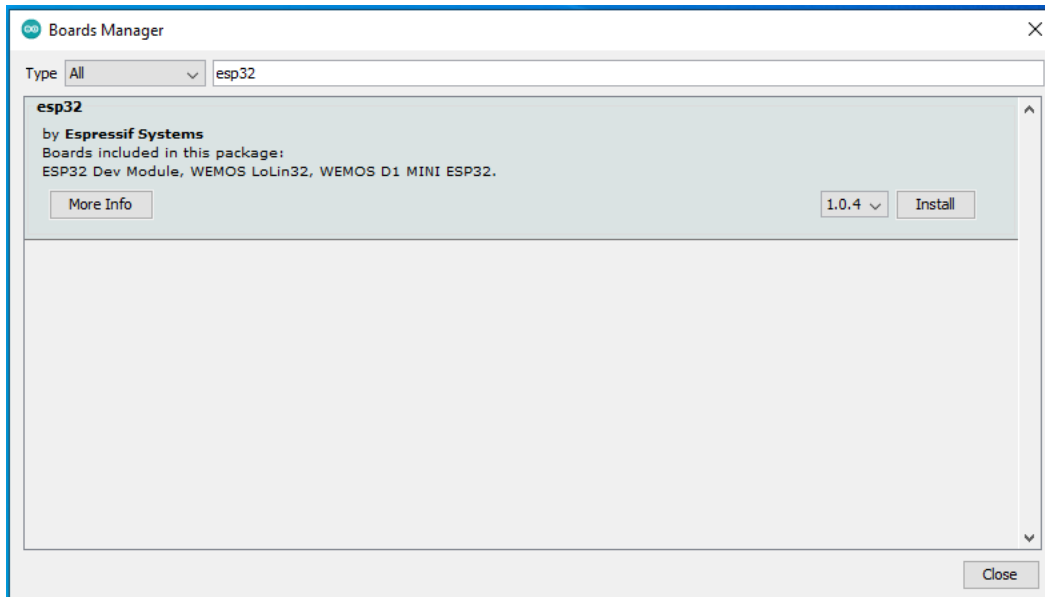
Puede mirar cómo quedará aquí abajo



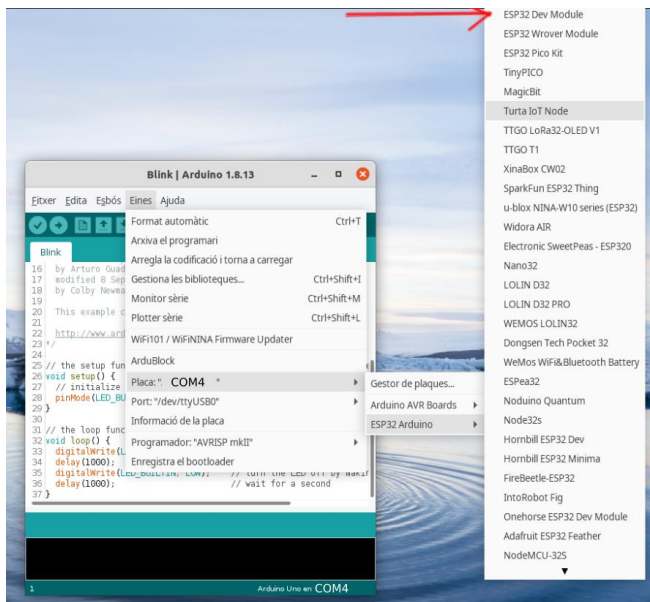
Ahora hay que ir a Herramientas
(Tools), Board: "Arduino uno", Boards Manager ... Pulse en Boards Manager ...



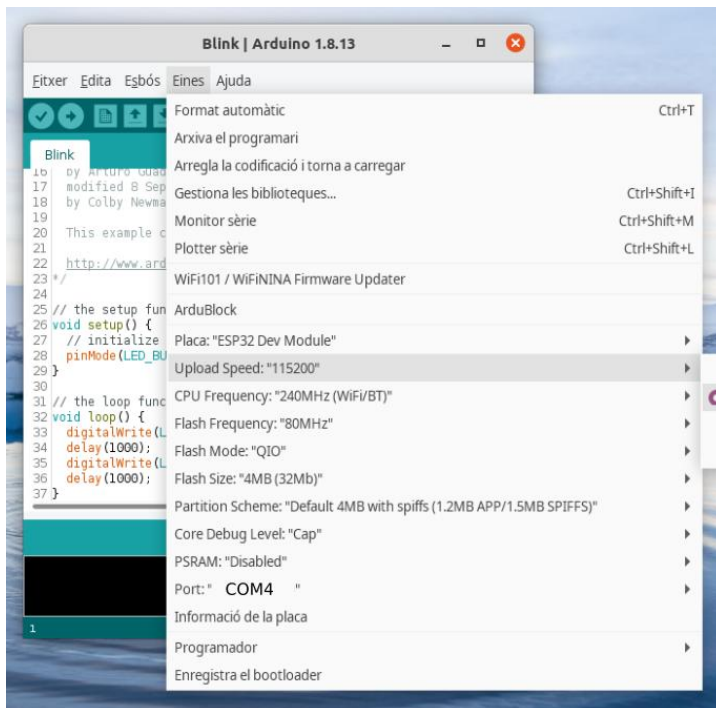
A continuación, hay que escribir en la ventana de búsqueda, buscar los controladores ESP32 e instalarlos.



Después vamos a Herramientas, Placa: Arduino, ESP32 Arduino, ESP32 Dev Module



Ponemos la velocidad de subida a 115200



Ahora abrimos los ejemplos que nos da Arduino IDE para poder comprobar el funcionamiento de las placas instaladas, en este caso es el de blink pero la adaptaremos para nuestra ESP32

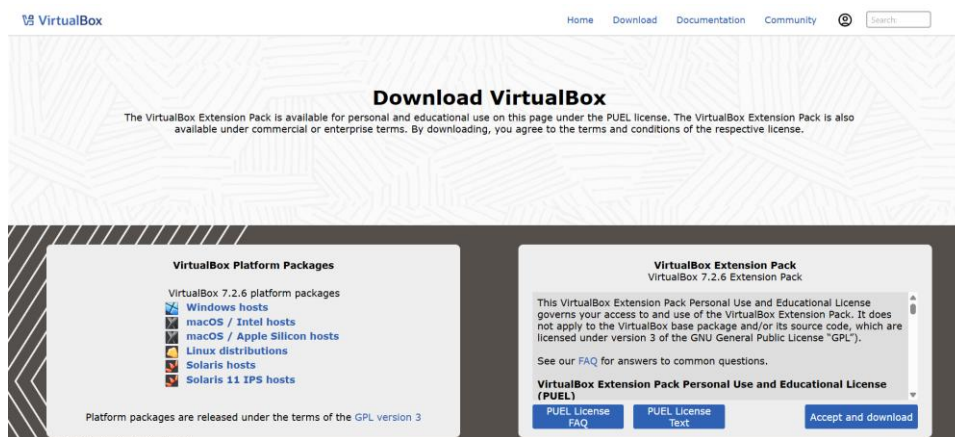
```
24
25 const int ledPin = 2;
26
27 // the setup function runs once when you press reset or power the board
28 void setup() {
29     Serial.begin(115200);
30     // initialize digital pin LED_BUILTIN as an output.
31     pinMode(ledPin, OUTPUT);
32 }
33
34 // the loop function runs over and over again forever
35 void loop() {
36     Serial.println("Hola Mundo")
37     digitalWrite(ledPin, HIGH); // turn the LED on (HIGH is the voltage level)
38     delay(1000);                // wait for a second
39     digitalWrite(ledPin, LOW);  // turn the LED off by making the voltage LOW
40     delay(1000);                // wait for a second
41 }
42
```

Instalación de Ubuntu Server en Máquina Virtual

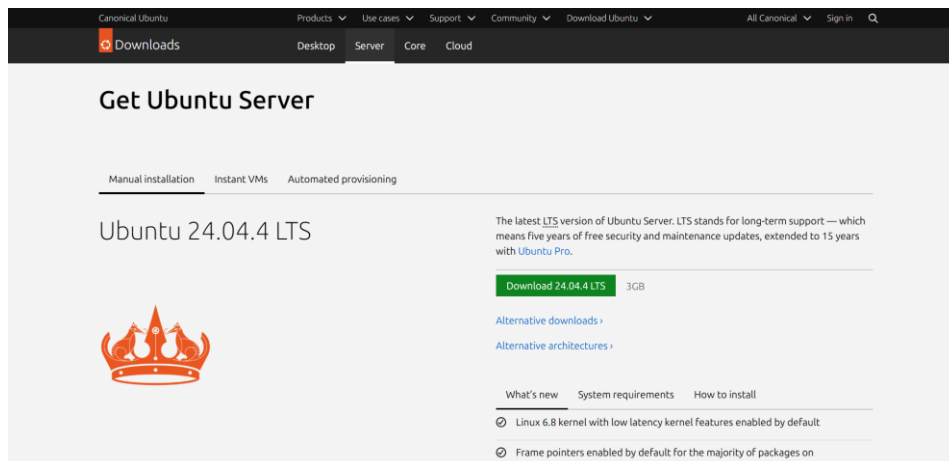
Para el desarrollo del proyecto se decidió utilizar un servidor basado en Ubuntu Server, instalado sobre una máquina virtual, con el fin de simular un entorno real de servidor sin necesidad de hardware dedicado. Esta decisión permite flexibilidad, facilidad de respaldo y control del entorno durante el desarrollo académico del sistema.

La máquina virtual fue creada utilizando un software de virtualización, al cual se le asignaron recursos básicos como memoria RAM, almacenamiento y adaptador de red en modo puente. Posteriormente, se realizó la instalación del sistema operativo Ubuntu Server siguiendo el asistente de instalación oficial.

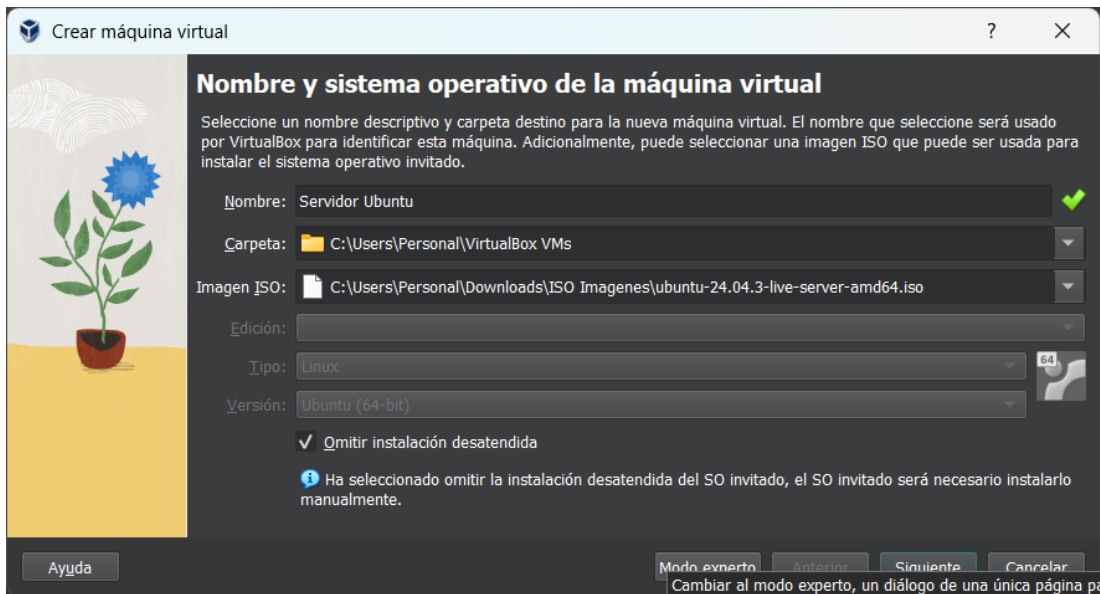
Para empezar, tendremos que tener un software de virtualización, como es el caso de VirtualBox, puede ingresar al siguiente enlace para obtenerlo <https://www.virtualbox.org/wiki/Downloads>



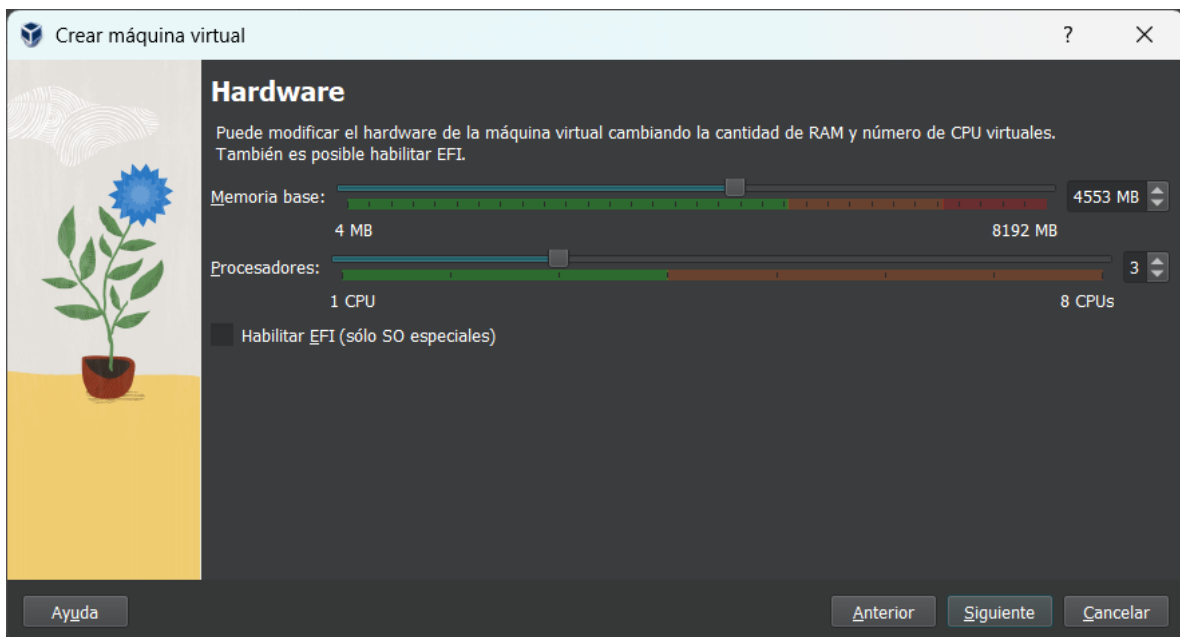
También deberemos contar con la ISO del sistema Ubuntu Server para poder empezar a instalarlo por VirtualBox, lo puedes encontrar en el siguiente enlace <https://ubuntu.com/download/server>



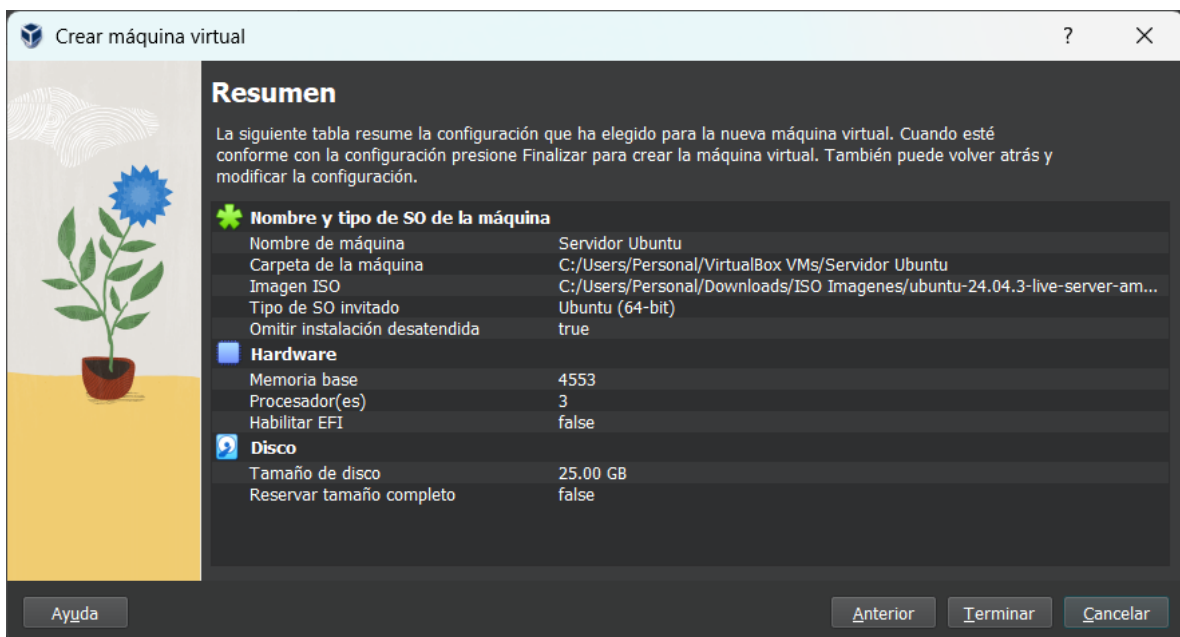
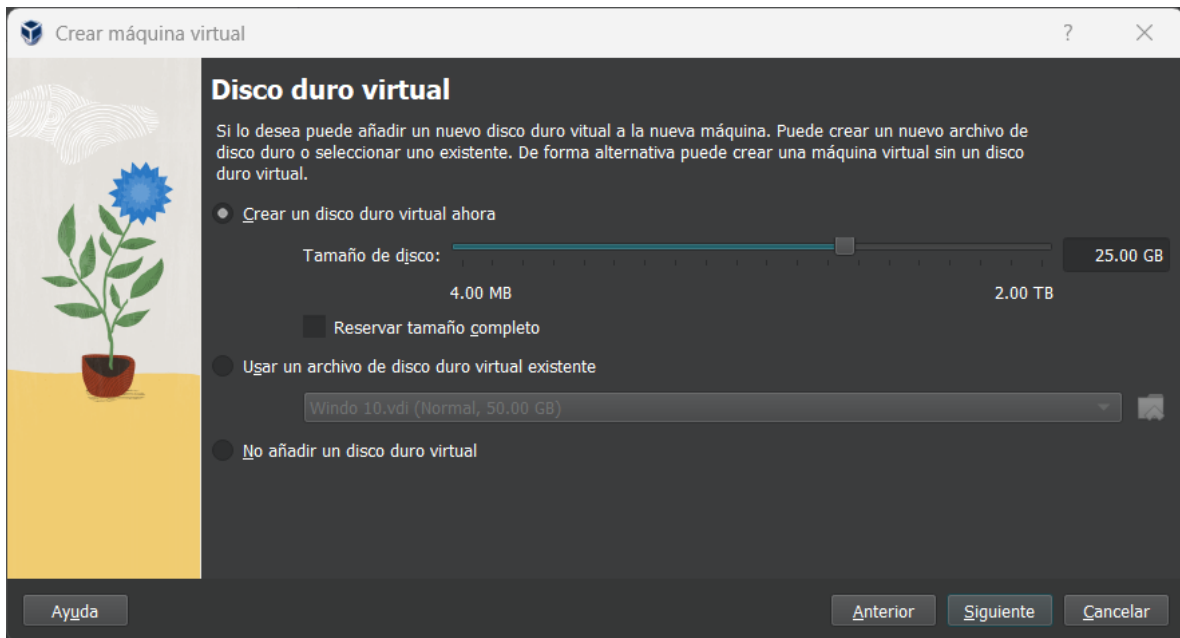
Ya teniendo instalado VirtualBox instalado y la ISO de Ubuntu Server procederemos a instalarlo dando clic en Nueva y se nos abrirá la siguiente ventana



Nos pedirá darle un nombre, esto es a consideración de nosotros yo le di ese, la imagen ISO seleccionamos el sistema que descargamos, y activamos la opción “Omitir instalación desatendida”, esto para ingresar nosotros mismo un usuario y contraseña, clic en siguiente

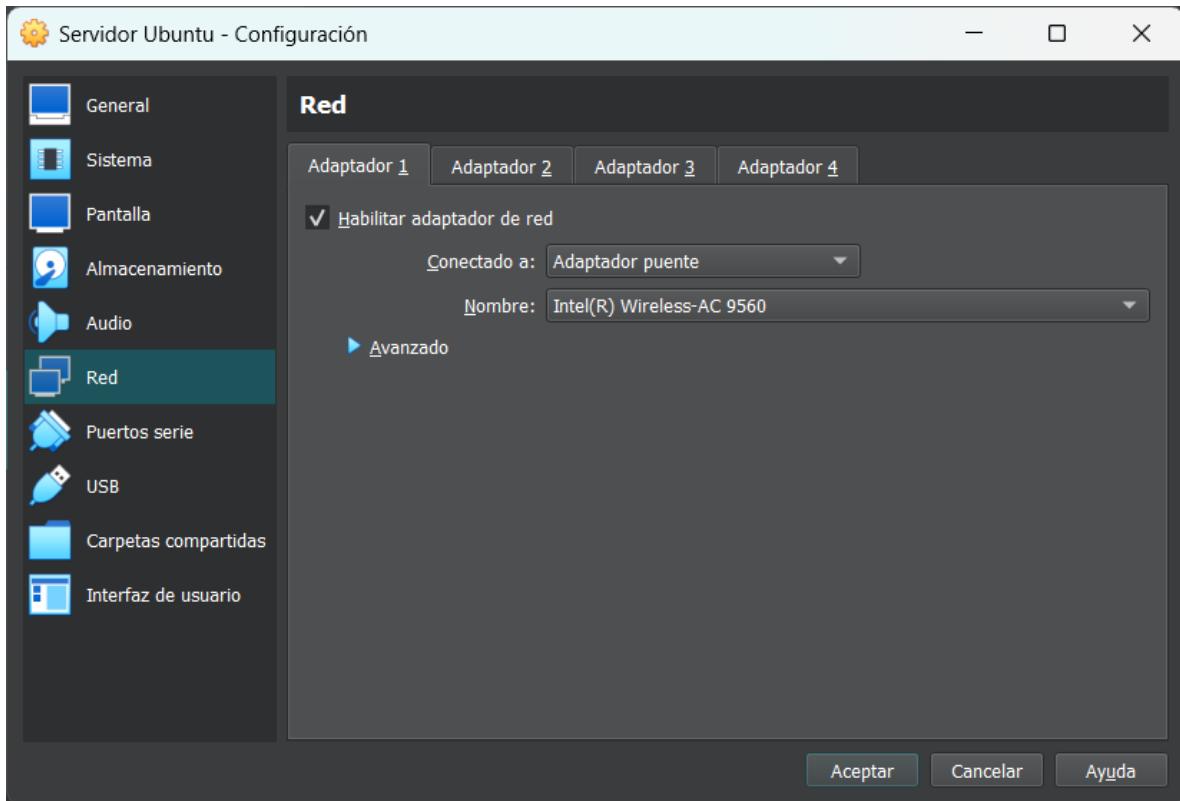


Aquí asignamos la memoria RAM y los procesadores esto a consideración de las especificaciones del equipo tenga, cuidado con asignarle mas de lo que tu equipo tiene actualmente

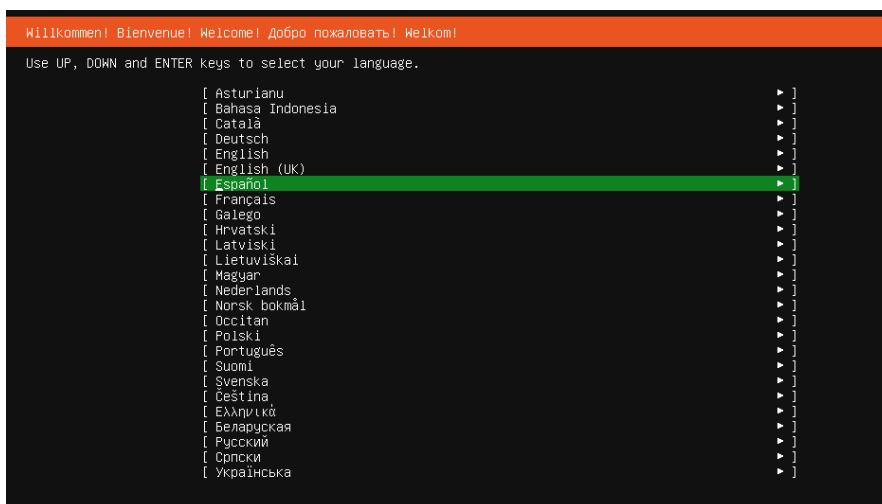


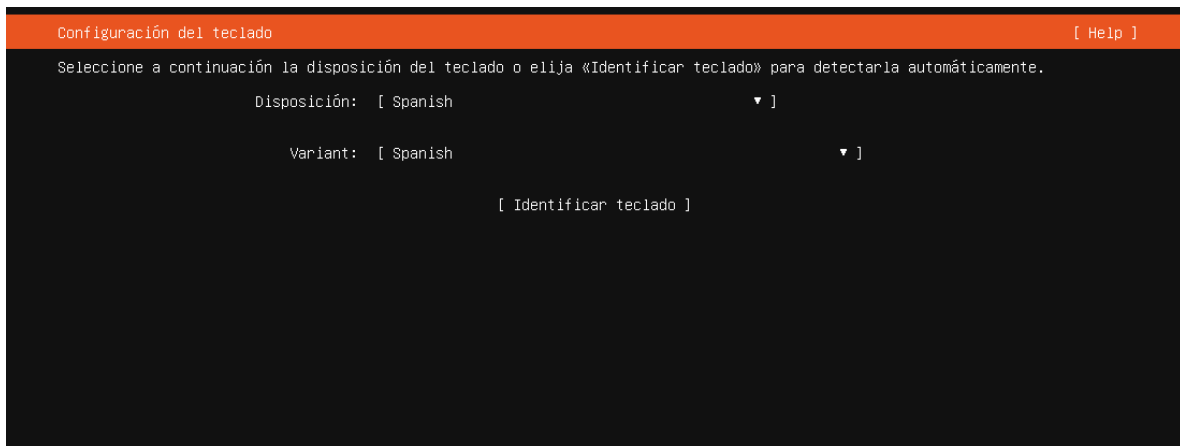
Nos pedirá asignar tamaño de disco, esto es a consideración, pueden asignarle el adecuado, siempre y cuando su equipo tenga espacio de sobra, damos clic y nos saldrá una ventana resumiendo las configuraciones que hemos hecho y damos en terminar

Ahora en la configuración de la maquina vamos en la parte de red y activamos el adaptador puente, esto permitirá compartir el internet que tengamos acceso y asignando una dirección a la maquina virtual y damos en aceptar, después de este paso iniciamos la maquina

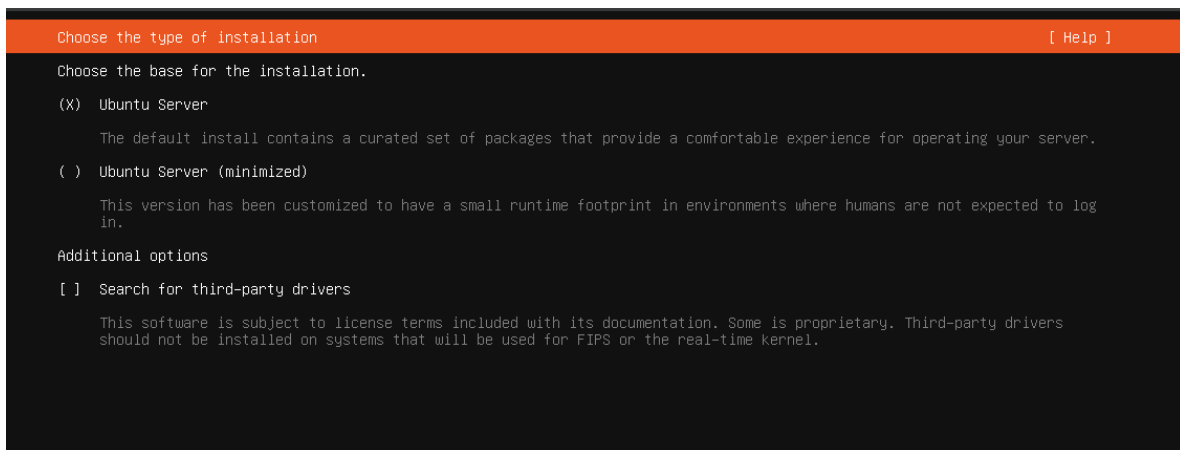


Iniciado una vez la maquina nos aparecerá primero la opción del idioma del sistema y del teclado, damos la de nuestra preferencia, cabe recalcar que para movernos por el asistente de instalación nos moveremos con las flechas de dirección y para aceptar con la tecla enter para “Hecho”

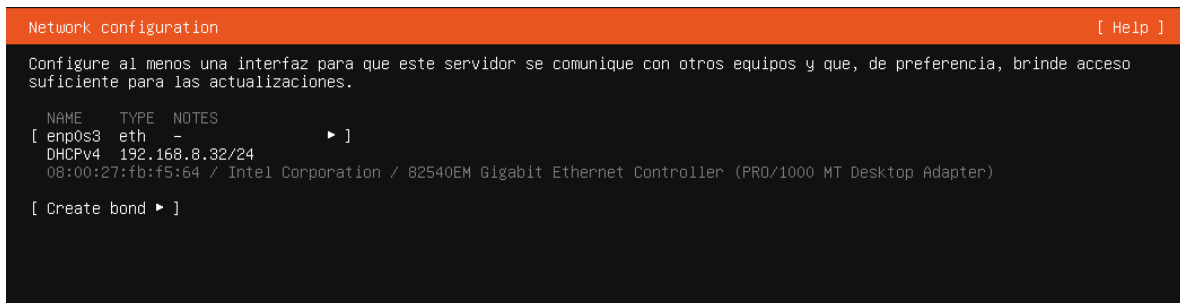




Para esta ventana usaremos la opción predeterminada, le damos en Hecho



En esta ventana aparece la configuración de la red, si recordamos activamos el adaptador puente y nos muestra ya una dirección relacionada a la de nuestra computadora principal, lo dejamos así por el momento y damos Hecho



Nos sale sobre asignarle un proxy al sistema, lo dejamos en blanco ya que no utilizaremos uno y le damos en hecho

Proxy configuration

[Help]

If this system requires a proxy to connect to the internet, enter its details here.

Proxy address:

If you need to use a HTTP proxy to access the outside world, enter the proxy information here. Otherwise, leave this blank.

The proxy information should be given in the standard form of "http://[[user][:pass]@host[:port)]/".

Para esta parte esta la dirección de donde se encuentra el directorio del gestor de paquetes de Ubuntu, por lo general se deja el predeterminado, damos en Hecho

If you use an alternative mirror for Ubuntu, enter its details here.

Mirror address:

You may provide an archive mirror to be used instead of the default.

The mirror location is being tested. \

Obj:1 http://mx.archive.ubuntu.com/ubuntu noble InRelease
Des:2 http://mx.archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Des:3 http://mx.archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]

Esta parte nos dice si queremos usar todo el disco para instalar Ubuntu, nos movemos con flechas y le damos enter a Hecho

Guided storage configuration

[Help]

Configure a guided storage layout, or create a custom one:

(X) Use an entire disk

[VBOX_HARDDISK_VBc27358ad-3833212b local disk 25.000G ▼]

[X] Set up this disk as an LVM group

[] Encrypt the LVM group with LUKS

Passphrase:

Confirm passphrase:

[] Also create a recovery key
The key will be stored as ~/recovery-key.txt in the live system and will be copied to /var/log/installer/ in the target system.

() Custom storage layout

Después de esto nos sale un resumen de las configuraciones que hemos realizado, también le damos a Hecho

```
RESUMEN DEL SISTEMA DE ARCHIVOS

PUNTO DE MONTAJE      TAMAÑO  TIPO      TIPO DE DISPOSITIVO
[ /                   11.496G new ext4   new LVM logical volume      ► ]
[ /boot              2.000G  new ext4   new partition of disco local ► ]

DISPOSITIVOS DISPONIBLES

DISPOSITIVO                                TIPO                                TAMAÑO
[ ubuntu-vg (new)                                LVM volume group                22.996G ► ]
espacio disponible                                11.500G ► ]

[ Create software RAID (md) ► ]
[ Crear grupo de volúmenes (LVM) ► ]

DISPOSITIVOS UTILIZADOS

DISPOSITIVO                                TIPO                                TAMAÑO
[ ubuntu-vg (new)                                LVM volume group                22.996G ► ]
ubuntu-lv                                new, to be formatted as ext4, mounted at / 11.496G ► ]

[ VBOX_HARDDISK_VBc27358ad-3833212b                disco local                    25.000G ► ]
partition 1                                new, BIOS grub spacer                1.000M ► ]
partition 2                                new, to be formatted as ext4, mounted at /boot 2.000G ► ]
partition 3                                new, PV of LVM volume group ubuntu-vg    22.997G ► ]
```

Una vez aceptado nos aparecerá un apartado de confirmar la instalación y damos en continuar

```
———— Confirmar acción destructiva ————

Selecting Continue below will begin the installation process and
result in the loss of data on the disks selected to be formatted.

You will not be able to return to this or a previous screen once the
installation has started.

Are you sure you want to continue?

[ No ]
[ Continuar ]
```

Ahora nos pedirá registrar un usuario, contraseña y nombre de servidor, esto es cuestión del usuario, completamos el formulario y damos enter a Hecho

Profile configuration [Help]

Enter the username and password you will use to log in to the system. You can configure SSH access on a later screen, but a password is still needed for sudo.

Su nombre:

Your servers name:
The name it uses when it talks to other computers.

Elija un nombre de usuario:

Elija una contraseña:

Confirme la contraseña:

Después de haber creado el usuario y el servidor, nos dirá si queremos activar Ubuntu pro, dejamos como esta la configuración y le damos en Hecho

Upgrade to Ubuntu Pro [Help]

Upgrade this machine to Ubuntu Pro for security updates on a much wider range of packages, until 2034. Assists with FedRAMP, FIPS, STIG, HIPAA and other compliance or hardening requirements.

[About Ubuntu Pro ►]

() Enable Ubuntu Pro

(X) Skip for now

You can always enable Ubuntu Pro later using the 'pro attach' command.

En esta ventana dirá si queremos instalar el servidor SSH, esto es a consideración pero yo lo activo para poder conectarme remotamente desde la terminal de Windows, una vez configurado este paso damos en Hecho

SSH configuration

You can choose to install the OpenSSH server package to enable secure remote access to your server.

[X] Instalar servidor OpenSSH

[X] Permitir autenticación con contraseña por SSH

[Import SSH key ►]

AUTHORIZED KEYS

No authorized key

En esta ventana nos dira si queremos instalar esta herramienta, es a consideración nuevamente, yo omito este paso ya que no requerimos ninguna de estas herramientas, bajamos y damos en Hecho

```
Featured server snaps [ Help ]

These are popular snaps in server environments. Select or deselect with SPACE, press ENTER to see more details of the package, publisher and versions available.

[ ] microk8s          canonical✓    Kubernetes for workstations and appliances
[ ] nextcloud         nextcloud✓   Nextcloud Server - A safe home for all your data
[ ] wekan             xet7        Open-Source Kanban
[ ] canonical-livepatch canonical✓    Canonical Livepatch Client
[ ] rocketchat-server rocketchat✓  Rocket.Chat server
[ ] mosquitto         mosquitto✓  Eclipse Mosquitto MQTT broker
[ ] etcd              canonical✓  Resilient key-value store by CoreOS
[ ] powershell        canonical✓  PowerShell for every system!
[ ] sabnzbd           safihre     SABnzbd - The automated Usenet download tool
[ ] wormhole          snapcrafters get things from one computer to another, safely
[ ] aws-cli           aws✓       Universal Command Line Interface for Amazon Web Services
[ ] google-cloud-sdk  google-cloud-sdk✓ Google Cloud SDK
[ ] slcli             softlayer   Python based SoftLayer API Tool.
[ ] doctl             digitalocean The official DigitalOcean command line interface
[ ] keepalived        keepalived-project High availability VRRP/BFD and load-balancing for Linux
[ ] prometheus        canonical✓  The Prometheus monitoring system and time series database
[ ] lxd               canonical✓  LXD - container and VM manager
```

Ahora empezara con a instalación, no desesperes, esto puede demorar y al terminar pedirá reiniciar para aplicar las configuraciones

```
Instalando el sistema [ Help ]

subiquity/Ad/apply_autoinstall_config:
subiquity/Late/apply_autoinstall_config:
configuring apt
  curtin command in-target
installing system
  executing curtin install initial step
  executing curtin install partitioning step
  curtin command install
    configuring storage
      running 'curtin block-meta simple'
      curtin command block-meta
        removing previous storage devices
        configuring disk: disk-sda
        configuring partition: partition-0
        configuring partition: partition-1
        configuring format: format-0
        configuring partition: partition-2
        configuring lvm_volgroup: lvm_volgroup-0
        configuring lvm_partition: lvm_partition-0
        configuring format: format-1
        configuring mount: mount-1
        configuring mount: mount-0
```

Una vez se haya reiniciado iniciaremos sesión con el usuario y contraseña que establecimos en la instalación

```
Ubuntu 24.04.3 LTS esmeraldaserver tty1

esmeraldaserver login: esmeralda
Password:
Welcome to Ubuntu 24.04.3 LTS (GNU/Linux 6.8.0-94-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of 2025-04-14:
lsb_release -a -f=24.04.3 LTS
```

Instalación de Docker en Ubuntu Server

Tras la configuración inicial de Ubuntu Server, se implementó Docker como plataforma de contenedores para gestionar aplicaciones de manera aislada y eficiente. La elección de esta herramienta se fundamenta en su robustez en entornos profesionales y su versatilidad para desplegar servicios críticos, como bases de datos y brokers de mensajería. El proceso se realizó vía terminal siguiendo los protocolos de estabilidad recomendados, finalizando con una prueba de ejecución para validar el estado operativo del motor.

Primero antes de instalar Docker es tener actualizado el sistema, por lo que estos comandos serán útiles:

- `sudo apt update`
- `sudo apt upgrade`

La documentación nos pide eliminar los paquetes que ofrecen algunas distribuciones de Linux para Docker, ya que estas no son oficiales de Docker y pueden entrar en conflicto con las oficiales

`sudo apt remove $(dpkg --get-selections docker.io docker-compose docker-compose-v2 docker-doc podman-docker containerd runc | cut -f1)`

```
~$ sudo apt remove $(dpkg --get-selections docker.io docker-compose docker-compose-v2 docker-doc podman-docker containerd runc | cut -f1)
dpkg: no se ha encontrado ningún paquete que corresponda con docker.io.
dpkg: no se ha encontrado ningún paquete que corresponda con docker-compose.
dpkg: no se ha encontrado ningún paquete que corresponda con docker-compose-v2.
dpkg: no se ha encontrado ningún paquete que corresponda con docker-doc.
dpkg: no se ha encontrado ningún paquete que corresponda con podman-docker.
dpkg: no se ha encontrado ningún paquete que corresponda con containerd.
dpkg: no se ha encontrado ningún paquete que corresponda con runc.
Leyendo lista de paquetes... Hecho
Creando árbol de dependencias... Hecho
Leyendo la información de estado... Hecho
0 actualizados, 0 nuevos se instalarán, 0 para eliminar y 2 no actualizados.
~$
```

Como se ve no se tienen ningún paquete instalado no oficial de Docker en el sistema

Ahora actualizamos la lista de paquetes de Ubuntu

`sudo apt update`

```
~$ sudo apt update
Obj:1 http://mx.archive.ubuntu.com/ubuntu noble InRelease
Des:2 http://mx.archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Des:3 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Des:4 http://mx.archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]
Des:5 http://mx.archive.ubuntu.com/ubuntu noble-updates/main amd64 Components [175 kB]
Des:6 http://mx.archive.ubuntu.com/ubuntu noble-updates/restricted amd64 Components [212 B]
Des:7 http://mx.archive.ubuntu.com/ubuntu noble-updates/universe amd64 Components [386 kB]
Des:8 http://mx.archive.ubuntu.com/ubuntu noble-updates/multiverse amd64 Components [940 B]
Des:9 http://mx.archive.ubuntu.com/ubuntu noble-backports/main amd64 Components [7.308 B]
Des:10 http://mx.archive.ubuntu.com/ubuntu noble-backports/restricted amd64 Components [216 B]
Des:11 http://mx.archive.ubuntu.com/ubuntu noble-backports/universe amd64 Components [10,5 kB]
Des:12 http://mx.archive.ubuntu.com/ubuntu noble-backports/multiverse amd64 Components [112 B]
```

Ahora nos pide instalar certificados SSL certificados para descargar e instalar archivos seguros desde internet

`sudo apt install ca-certificates curl`

```
$ sudo apt install ca-certificates curl
Leyendo lista de paquetes... Hecho
Creando árbol de dependencias... Hecho
Leyendo la información de estado... Hecho
ca-certificates ya está en su versión más reciente (20240203).
fijado ca-certificates como instalado manualmente.
curl ya está en su versión más reciente (8.5.0-2ubuntu10.6).
fijado curl como instalado manualmente.
0 actualizados, 0 nuevos se instalarán, 0 para eliminar y 2 no actualizados.
```

Ahora añadiremos la clave GPG de Docker oficial para en futura actualizaciones descargar una actualización original de Docker y no de fuentes de terceros

sudo install -m 0755 -d /etc/apt/keyrings

```
$ sudo install -m 0755 -d /etc/apt/keyrings
$
```

sudo curl -fsSL <https://download.docker.com/linux/ubuntu/gpg> -o /etc/apt/keyrings/docker.asc

```
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/docker.asc
```

sudo chmod a+r /etc/apt/keyrings/docker.asc

```
sudo chmod a+r /etc/apt/keyrings/docker.asc
```

Ahora agregaremos a la lista de paquetes de Ubuntu Server la dependencia donde se descargara Docker

sudo tee /etc/apt/sources.list.d/docker.sources <<EOF

Types: deb

URIs: <https://download.docker.com/linux/ubuntu>

Suites: \$(. /etc/os-release && echo "\${UBUNTU_CODENAME:-\$VERSION_CODENAME}")

Components: stable

Signed-By: /etc/apt/keyrings/docker.asc

EOF

Antes de agregar:

```
~$ sudo tee /etc/apt/sources.list.d/docker.sources <<EOF
Types: deb
URIs: https://download.docker.com/linux/ubuntu
Suites: $(. /etc/os-release && echo "${UBUNTU_CODENAME:-$VERSION_CODENAME}")
Components: stable
Signed-By: /etc/apt/keyrings/docker.asc
EOF
```

Después de agregar:

```
sudo tee /etc/apt/sources.list.d/docker.sources <<EOF
Types: deb
URIs: https://download.docker.com/linux/ubuntu
Suites: $(. /etc/os-release && echo "${UBUNTU_CODENAME:-$VERSION_CODENAME}")
Components: stable
Signed-By: /etc/apt/keyrings/docker.asc
EOF
Types: deb
URIs: https://download.docker.com/linux/ubuntu
Suites: noble
Components: stable
Signed-By: /etc/apt/keyrings/docker.asc
```

Ahora actualizamos la lista de paquetes donde ya aparece Docker

sudo apt update

```
sudo apt update
Obj:1 http://mx.archive.ubuntu.com/ubuntu noble InRelease
Obj:2 http://mx.archive.ubuntu.com/ubuntu noble-updates InRelease
Obj:3 http://mx.archive.ubuntu.com/ubuntu noble-backports InRelease
Obj:4 http://security.ubuntu.com/ubuntu noble-security InRelease
Des:5 https://download.docker.com/linux/ubuntu noble InRelease [48,5 kB]
Des:6 https://download.docker.com/linux/ubuntu noble/stable amd64 Packages [43,3 kB]
Descargados 91,8 kB en 4s (22,1 kB/s)
Leyendo lista de paquetes... Hecho
Creando árbol de dependencias... Hecho
Leyendo la información de estado... Hecho
Se pueden actualizar 2 paquetes. Ejecute «apt list --upgradable» para verlos.
```

Ya habiendo configurado la lista de dependencias para Docker oficial pasaremos a instalarlo con el siguiente comando

**sudo apt install docker-ce docker-ce-cli containerd.io docker-buildx-plugin
docker-compose-plugin**

```

sudo apt install docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin
Leyendo lista de paquetes... Hecho
Creando árbol de dependencias... Hecho
Leyendo la información de estado... Hecho
Se instalarán los siguientes paquetes adicionales:
  docker-ce-rootless-extras libslirp0 pigz slirp4netns
Paquetes sugeridos:
  cgroupfs-mount | cgroup-lite docker-model-plugin
Se instalarán los siguientes paquetes NUEVOS:
  containerd.io docker-buildx-plugin docker-ce docker-ce-cli docker-ce-rootless-extras docker-compose-plugin libslirp0 pigz slirp4netns
0 actualizados, 9 nuevos se instalarán, 0 para eliminar y 2 no actualizados.
Se necesita descargar 96,7 MB de archivos.
Se utilizarán 391 MB de espacio de disco adicional después de esta operación.
¿Desea continuar? [S/n] s
Des:1 https://download.docker.com/linux/ubuntu noble/stable amd64 containerd.io amd64 2.2.1-1-ubuntu.24.04-noble [23,4 MB]
Des:2 http://mx.archive.ubuntu.com/ubuntu noble/universe amd64 pigz amd64 2.8-1 [65,6 kB]

```

Ahora verificamos si el servicio de Docker esta esta activo con el siguiente comando

sudo systemctl status docker

Va ver mucho texto y no los dejara salir de este comando para introducir otro comando, solamente dan Ctrl C para cancelar el comando ejecutándose y poder introducir el siguiente

```

sudo systemctl status docker
● docker.service - Docker Application Container Engine
   Loaded: loaded (/usr/lib/systemd/system/docker.service; enabled; preset: enabled)
   Active: active (running) since Sun 2026-02-01 08:57:41 CST; 2min 23s ago
 TriggeredBy: ● docker.socket
    Docs: https://docs.docker.com
   Main PID: 3600 (dockerd)
     Tasks: 10
    Memory: 26.8M (peak: 27.8M)
       CPU: 4.371s
    CGroup: /system.slice/docker.service
            └─3600 /usr/bin/dockerd -H fd:// --containerd=/run/containerd/containerd.sock

```

Nota: en caso de no tenerlo activado se usará el siguiente comando para activarlo:

sudo systemctl start docker

Por último, paso verificamos si fue correcto la instalación ejecutando el ejemplo de hellow world

sudo docker run hello-world

```

sudo docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
17eec7bbc9d7: Pull complete
ea52d2000f90: Download complete
Digest: sha256:05813aedc15fb7b4d732e1be879d3252c1c9c25d885824f6295cab4538cb85cd
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (amd64)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.

```

Capítulo 2

Implementacion de contenedores y infraestructura

`docker --version`

`docker compose version`

```
Last login: Sat Feb 21 03:13:27 2026 from 192.168.161.4
bauti@bauti:~$ docker --version
Docker version 29.2.1, build a5c7197
bauti@bauti:~$ docker compose version
Docker Compose version v5.0.2
bauti@bauti:~$ |
```

Si nos aparece de esta manera es que lo tenemos correctamente instalado.

Ahora para una mejor organizacion en nuestros contenedores hacemos lo siguiente creamos un nuevo directorio donde gurdaremos la parte de la configuracion de nuestros contenedores.

`mkdir mi_stack_iot`

`cd mi_stack_iot`

Creamos el archivo `.yaml`

`nano docker-compose.yaml`

Se te abrirá el editor de texto nano en la terminal.

Y en este pegamos o escribimos el contenido del archivo para cargar los contenedores en este se encuentra el puerto nombre del contenedor entre otras configuraciones.

services:

mosquitto:

image: eclipse-mosquitto:latest

container_name: mosquitto

ports:

- "1883:1883"

- "9001:9001"

restart: unless-stopped

influxdb:

image: influxdb:latest

container_name: influxdb

ports:

- "8086:8086"

environment:

- DOCKER_INFLUXDB_INIT_MODE=setup

- DOCKER_INFLUXDB_INIT_USERNAME=admin

- DOCKER_INFLUXDB_INIT_PASSWORD=adminpassword

- DOCKER_INFLUXDB_INIT_ORG=mi_org

- DOCKER_INFLUXDB_INIT_BUCKET=mi_bucket

restart: unless-stopped

grafana:

image: grafana/grafana:latest

container_name: grafana

ports:

- "3000:3000"

restart: unless-stopped

Para guardar los cambios, presionamos Ctrl + O (la letra o), luego presionamos Enter, y por último Ctrl + X para salir del editor.

Una vez realizado esto levantamos los contenedores en este caso vamos a ejecutar el siguiente comando.

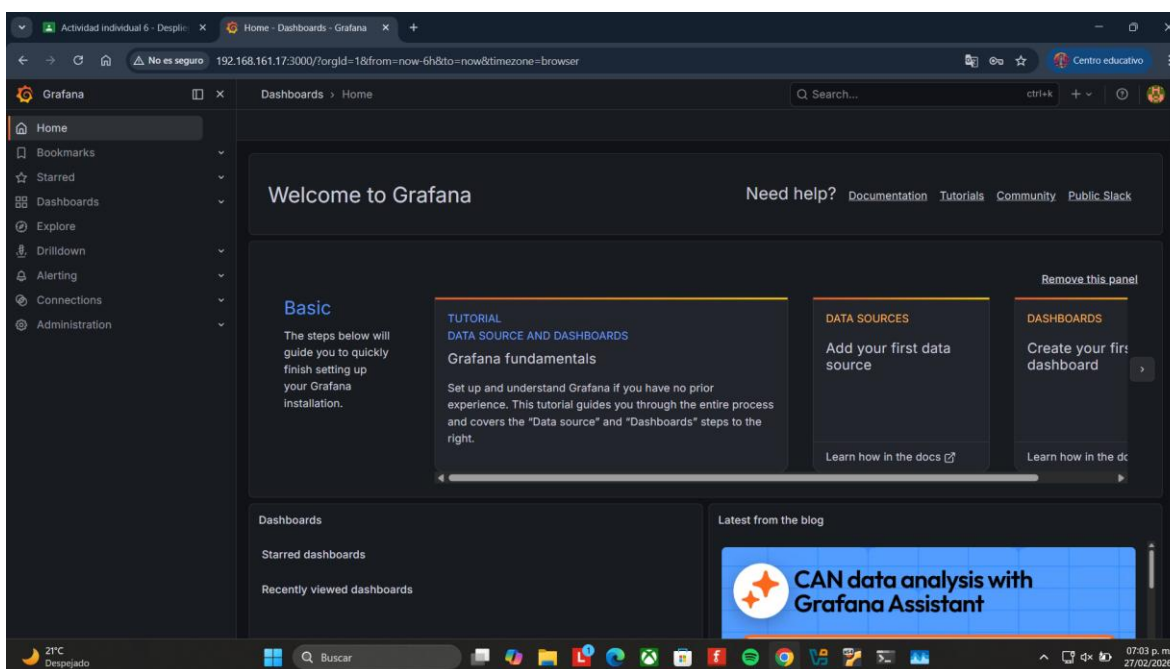
docker compose up -d

```
unable to get image 'eclipse-mosquitto:latest': permission denied
http://%2Fvar%2Frun%2Fdocker.sock/v1.51/images/eclipse-mosquitto:latest
bauti@bauti:~/mi_stack_iot$ sudo docker compose up -d
[sudo] password for bauti:
[+] up 10/28
  Image grafana/grafana:latest [  ] Pulling
  Image eclipse-mosquitto:latest [  ] Pulling
  Image influxdb:latest [  ] Pulling
```

Una vez hecho esto vamos a poder ver los contenedores arriba como se observa en la siguiente imagen.

```
bauti@bauti:~/mi_stack_iot$ sudo docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS
d8a0a18e630d   eclipse-mosquitto:latest            "/docker-entrypoint..." About a minute ago Up 59 seconds  0.0.0.0:1883->1883/tcp, [::]:1883->1883/tcp, 0.0.0.0
b63060f19d5b   influxdb:latest                    "/entrypoint.sh infl..." About a minute ago Up 59 seconds  0.0.0.0:8086->8086/tcp, [::]:8086->8086/tcp
e872c7fff533   grafana/grafana:latest              "/run.sh"                About a minute ago Up 59 seconds  0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp
```

Basicamente ya estarian disponibles nuestros contenedores en este caso vamos a comprobar esta parte vamos a la parte de grafana .



Y como vemos se levanta el servicio esto quiere decir que se completo la parte del levantamiento de los contenedores.

En resumen la infraestructura es la siguiente:

1. La Base: Docker y Docker Compose

En lugar de instalar los programas sueltos en tu Ubuntu (lo que puede generar conflictos de versiones), usamos Docker.

- Docker empaqueta cada servicio en su propio "contenedor" aislado.

- Usamos Docker Compose, que funciona como una lista de instrucciones (el archivo .yaml que creamos). Estas instrucciones le dice a Ubuntu: *"Descarga estos tres programas, enciéndelos al mismo tiempo y conéctalos en la misma red virtual para que puedan hablar entre ellos"*.

2. Mosquitto (El Broker MQTT) - *El que recibe*

En el mundo del Internet de las Cosas (IoT), los sensores (como los que miden temperatura o humedad) son pequeñitos y no tienen mucha potencia. Por eso se usa un lenguaje muy ligero llamado MQTT.

- Mosquitto actúa como un Broker (un intermediario o director de tráfico).
- Su único trabajo es recibir los mensajes de los sensores a toda velocidad y pasárselos al siguiente sistema. No guarda nada de forma permanente, solo recibe y pasa el mensaje.

3. InfluxDB (La Base de Datos de Series de Tiempo) - *El que guarda*

Aquí es donde se almacena la información, pero no es una base de datos normal como las de un banco o una tienda. Es una TSDB (Time-Series Database).

- Está diseñada específicamente para guardar eventos que dependen del tiempo.
- Cada vez que recibe un dato, le estampa la fecha y hora exacta (el *Timestamp*). Como está optimizada para esto, puede guardar miles de lecturas de sensores por segundo sin trabarse.

4. Grafana (La Interfaz Gráfica) - *El que muestra*

Grafana es la cara visual del proyecto.

- Por sí solo, Grafana está vacío y no almacena información. Lo que hace es conectarse a InfluxDB (actuando como su *Data Source* o fuente de datos).
- Cuando entramos a Grafana desde el navegador web, este extrae los números fríos de InfluxDB y los traduce al instante en gráficas, velocímetros y paneles interactivos muy visuales.

Capítulo 3

Herramienta utilizada

Las contraseñas se generaron usando la herramienta `mosquitto_passwd` incluida en la imagen oficial de Eclipse Mosquitto. Esta herramienta aplica un algoritmo de hashing PBKDF2-SHA512 que hace que las contraseñas sean computacionalmente muy costosas de revertir por fuerza bruta.

Preparacion del entorno

Antes de crear los usuarios se preparo el archivo que almacenara las credenciales:

```
touch passwordfile
```

```
administrator@EcoControl:~/ecocontrol$ touch passwordfile
```

```
touch aclfile
```

```
administrator@EcoControl:~/ecocontrol$ touch aclfile
```

Nota: El archivo debe existir previamente en el host para que Docker lo monte correctamente como archivo y no como directorio.

Comandos de creacion de usuarios

Se ejecutaron los siguientes comandos para crear cada usuario. La flag `-c` (create) no se utiliza porque requiere que el archivo no exista; en su lugar se usa el modo de adición directa:

Usuario telegraf (lector/recolector de datos):

```
sudo docker run --rm -it \
```

```
-v $(pwd)/passwordfile:/mosquitto/config/passwordfile \
```

```
eclipse-mosquitto \
```

```
mosquitto_passwd /mosquitto/config/passwordfile telegraf
```

```
administrator@EcoControl:~/ecocontrol$ sudo docker run --rm -it \
-v $(pwd)/passwordfile:/mosquitto/config/passwordfile \
eclipse-mosquitto \
mosquitto_passwd /mosquitto/config/passwordfile telegraf
```

Usuarios de los dispositivos ESP32:

```
-v $(pwd)/passwordfile:/mosquitto/config/passwordfile \
```

```
eclipse-mosquitto \
```

```
mosquitto_passwd /mosquitto/config/passwordfile user_esmeralda
```

```
administrator@EcoControl:~/ecocontrol$ sudo docker run --rm -it \
-v $(pwd)/passwordfile:/mosquitto/config/passwordfile \
eclipse-mosquitto \
mosquitto_passwd /mosquitto/config/passwordfile user_esmeralda
```

Se repitió el mismo comando para:

- user_cristian
- user_alexis
- user_miguel
- user_kevin

Verificación de usuarios creados

Las contraseñas se almacenan únicamente en forma hasheada. Para verificar que los usuarios fueron creados correctamente:

```
cat passwordfile
```

```
administrator@EcoControl:~/ecocontrol$ cat passwordfile
telegraf:$7$1000$+zCxmo68srUYZjgsViihbFbLunMH1l+IqOYd6YF
h4AJ+400abkzrDODDB+oMni6l1ppYRJJBWxjHBA1jSn/t8Zq4AcwpDZA
user_cristian:$7$1000$I/zoZwmqYwQkCOiDgwbGRzbW57RbVI9reZ
Wu0RsLG+EhqWqCl9s2ZJhIVZE+lWbDFTAtV7CpWVcLRIWORDXNzWsbjZ
user_esmeralda:$7$1000$koz/K9TaqLkVfPKTCDF53QA0YN5V4pm24
zox/qUzaAKr1rlzHE2G1zCo+XIUEjoWi5iX013GGEiDdzo0Cnfpr2NU8
user_alexis:$7$1000$B1B3QMQGLRe00/2eYubqLIE2crC0UnkBiws7
g1vQ4cFskKOnmLsFEFNesKXWtaI99SFCV6yJkp5CqN1gQ9Ia2meyRPT4
user_miguel:$7$1000$D4sW9GHFlcCDGcSSR62GqLt0/jxQvPYqvdJl
hxonufvm4yo0WL4AeYpQ9i13A2Y6NnARcA7p2PI498Dy1HreCjyLdmvS
user_kevin:$7$1000$gXDhRz3PfcZpzd0xEudU4AWfJHchNJdgZ0YkE
QqZjI5gtz6R25uDgtWBRA46v0o9Gs8Juh2NK2kPKn6e6LRl3WWceD+wE
```

Las contraseñas en texto plano nunca se almacenan en el sistema, solo el administrador las conoce al momento de crearlas.

Ajuste de permisos del archivo de contraseñas

El proceso de Mosquitto dentro del contenedor corre como el usuario mosquitto (no root), por lo que el archivo de contraseñas necesita permisos de lectura adecuados:

```
sudo chmod 0644 passwordfile
```

```
administrator@EcoControl:~/ecocontrol$ sudo chmod 0644 passwordfile
```

Verificacion:

```
ls -la passwordfile
```

```
-rw-r--r-- 1 root root 1189 passwordfile
```

```
administrator@EcoControl:~/ecocontrol$ ls -la passwordfile  
-rw-r--r-- 1 root root 1189 Mar 20 04:35 passwordfile
```

Configuracion del Broker Mosquitto

mosquitto.conf

Se modifico el archivo de configuracion principal para deshabilitar el acceso anonimo y requerir autenticacion:

```
administrator@EcoControl:~/ecocontrol$ sudo nano mosquitto.conf  
[sudo] password for administrator:
```

```
GNU nano 7.2  
listener 1883  
allow_anonymous false  
password_file /mosquitto/config/passwordfile  
acl_file /mosquitto/config/aclfile
```

listener 1883

allow_anonymous false

password_file /mosquitto/config/passwordfile

acl_file /mosquitto/config/aclfile

Directiva	Valor	Descripcion
listener	1883	Puerto estandar MQTT
allow_anonymous	false	Rechaza conexiones sin credenciales
password_file	/mosquitto/config/passwordfile	Archivo con usuarios y hashes
acl_file	/mosquitto/config/aclfile	Archivo con reglas de acceso por topico

Lista de Control de Acceso (ACL)

Se implementaron reglas de minimo privilegio: cada usuario solo puede acceder a los topicos estrictamente necesarios para su funcion.

```
administrator@EcoControl:~/ecocontrol$ sudo nano aclfile
```

```
GNU nano 7.2
# Telegraf puede leer todo
user telegraf
topic readwrite esp32/ecocontrol/#

# Kevin
user user_kevin
topic write esp32/ecocontrol/kevin/#

# Cristian
user user_cristian
topic write esp32/ecocontrol/cristian/#

# Esmeralda
user user_esmeralda
topic write esp32/ecocontrol/EsmeTrejo/#

# Miguel
user user_miguel
topic write esp32/ecocontrol/Migue/#

# Alexis
user user_alexis
topic write esp32/ecocontrol/alexis/#
```

Comando a pegar:

Telegraf puede leer todo

user telegraf

topic readwrite esp32/ecocontrol/#

Kevin

user user_kevin

topic write esp32/ecocontrol/kevin/#

Cristian

user user_cristian

topic write esp32/ecocontrol/cristian/#

Esmeralda

user user_esmeralda

topic write esp32/ecocontrol/EsmeTrejo/#

Miguel

user user_miguel

topic write esp32/ecocontrol/Migue/#

Alexis

user user_alexis

topic write esp32/ecocontrol/alexis/#

Montaje de volúmenes en Docker

Es crítico montar los tres archivos de configuración como volúmenes en el docker-compose.yml para que Mosquitto los encuentre dentro del contenedor. Sin estos volúmenes el broker no puede leer las credenciales ni las reglas ACL:

mosquitto:

image: eclipse-mosquitto

container_name: mosquitto

restart: always

ports:

- "1883:1883"

volumes:

- ./mosquitto.conf:/mosquitto/config/mosquitto.conf

- ./passwordfile:/mosquitto/config/passwordfile

- ./aclfile:/mosquitto/config/aclfile

```
mosquitto:
  image: eclipse-mosquitto
  container_name: mosquitto
  restart: always
  ports:
    - "1883:1883"
  volumes:
    - ./mosquitto.conf:/mosquitto/config/mosquitto.conf
    - ./passwordfile:/mosquitto/config/passwordfile
    - ./aclfile:/mosquitto/config/aclfile
```

Aplicación de cambios

Para que los cambios en volúmenes y configuración surtan efecto, se requiere bajar y volver a levantar los contenedores (un simple restart no recarga los volúmenes):

sudo docker compose down

sudo docker compose up -d

```

administrator@EcoControl:~/ecocontrol$ sudo docker compose down
[+] down 5/5
✓Container grafana          Removed
✓Container telegraf         Removed
✓Container mosquitto        Removed
✓Container influxdb         Removed
✓Network ecocontrol_default Removed
administrator@EcoControl:~/ecocontrol$ sudo docker compose up -d
[+] up 5/5
✓Network ecocontrol_default Created
✓Container influxdb          Started
✓Container mosquitto         Started
✓Container grafana           Started
✓Container telegraf          Started
administrator@EcoControl:~/ecocontrol$ sudo docker ps

```

Reglas de Firewall con UFW

Politica general

El firewall esta configurado con una politica de denegacion por defecto para trafico entrante, lo que significa que cualquier puerto que no este explicitamente permitido queda bloqueado:

sudo ufw status verbose

```

administrator@EcoControl:~$ sudo ufw status verbose
[sudo] password for administrator:
Status: active
Logging: on (low)
Default: deny (incoming), allow (outgoing), deny (routed)
New profiles: skip

To Action From
--
22 ALLOW IN Anywhere
3000/tcp ALLOW IN Anywhere
3001/tcp ALLOW IN Anywhere
1883/tcp ALLOW IN Anywhere
5678/tcp ALLOW IN Anywhere
9443/tcp ALLOW IN Anywhere
22 (v6) ALLOW IN Anywhere (v6)
3000/tcp (v6) ALLOW IN Anywhere (v6)
3001/tcp (v6) ALLOW IN Anywhere (v6)
1883/tcp (v6) ALLOW IN Anywhere (v6)
5678/tcp (v6) ALLOW IN Anywhere (v6)
9443/tcp (v6) ALLOW IN Anywhere (v6)

```

Politica	Trafico	Efecto
deny (incoming)	Entrante	Bloquea todo lo que no este en la lista de permitidos
allow (outgoing)	Saliente	El servidor puede hacer peticiones hacia el exterior
deny (routed)	Enrutado	No se permite el reenvio de paquetes

Puertos habilitados

Puerto	Protocolo	Servicio	Justificacion
22	TCP	SSH	Acceso remoto para administracion del servidor
1883	TCP	MQTT (Mosquitto)	Comunicacion entre ESP32 y el broker de mensajes
3000	TCP	Grafana	Dashboard web de visualizacion de datos de sensores
3001	TCP	Open WebUI	Interfaz web para modelos de IA locales
5678	TCP	n8n	Plataforma de automatizacion y envio de alertas
9443	TCP	Portainer	Panel de administracion de contenedores Docker (HTTPS)

Comandos utilizados para configurar el firewall

Para consultar el estado actual del firewall:

```
sudo ufw status verbose
```

```

administrator@EcoControl:~$ sudo ufw status verbose
[sudo] password for administrator:
Sorry, try again.
[sudo] password for administrator:
Status: active
Logging: on (low)
Default: deny (incoming), allow (outgoing), deny (routed)
New profiles: skip

```

To	Action	From
--	-----	----
22	ALLOW IN	Anywhere
3000/tcp	ALLOW IN	Anywhere
3001/tcp	ALLOW IN	Anywhere
1883/tcp	ALLOW IN	Anywhere
5678/tcp	ALLOW IN	Anywhere
9443/tcp	ALLOW IN	Anywhere
22 (v6)	ALLOW IN	Anywhere (v6)
3000/tcp (v6)	ALLOW IN	Anywhere (v6)
3001/tcp (v6)	ALLOW IN	Anywhere (v6)
1883/tcp (v6)	ALLOW IN	Anywhere (v6)
5678/tcp (v6)	ALLOW IN	Anywhere (v6)
9443/tcp (v6)	ALLOW IN	Anywhere (v6)

Para habilitar un puerto (ejemplo con MQTT):

```
sudo ufw allow 1883/tcp
```

```

administrator@EcoControl:~$ sudo ufw allow 1883/tcp
Rule added
Rule added (v6)

```

Para verificar que una regla específica está activa:

```
sudo ufw status | grep 1883
```

```

administrator@EcoControl:~$ sudo ufw status | grep 1883
1883/tcp          ALLOW          Anywhere
1883/tcp (v6)     ALLOW          Anywhere (v6)

```